

Initial Understanding:

My initial understanding of this project was that we have been provided the starting of the dataset. Then we have to give prompt to the model and then the model acts as a suspect for bunch of output. Then we have to finetune the model to extract the reliable information from the model and after fine tuning we need to build the whole pipeline .

Why Structured Facts Instead of Raw Text :

After reading the first doubt that came in my mind was that they expect model to be auditable for the reasoning. So every conclusion should be traceable. So we basically have to work in the way the actual detective works-they gather testimony, look for inconsistencies, pressure the most suspicious people, and stop when the evidence is clear enough.

If we directly give the raw text to the LLM and ask to predict the murderer, it can do it but it will not be able to explain why it reached to this conclusion . So just analysing raw text wont work, we need to work upon the [facts](#). We need structured data, we can't work on long [paragraphs](#). So the main task is to extract the facts from the given [data](#). So basically everytime we do that we need to extract the main info like location, time at which that person was at that [location](#). So basically we need info like was_at, was_with, knew some information and all such kind of things.

Why to use an LLM for Extraction :

So next thing is why do we use LLMs for this. So basically everytime a suspect gives a statement, the format of speaking of all the suspects might not be same. So we use LLM for [extraction](#). We basically ask LLM to output a json array where the predicate might be one from the given set and confidence must be float. LLM just extracts the facts. The reasoning is done based on the set of rules you have to the LLM.

Why FineTuning:

Finetuning is basically teaching a small local model your specific task so you don't have to explain it every time and can run it for free without internet. And main reason of fine tuning is every call to Groq costs tokens and adds latency. If you have a local fine-tuned model that already knows how to output your specific JSON scheme, you don't need to explain the scheme in every prompt. That reduces token cost and speeds up inference. I used TinyLlama because it has less parameters and is small enough to run on T4 of colab. I used LoRA (Low Rank Adaptation) instead of full finetuning because tuning all parameters will cause a lot of memory usage. LoRA basically adds small matrices to specific attention layers.

Why Additive Scoring Instead of Bayesian :

I used started with Bayesian scoring initially i.e. assuming that all the suspects have uniform prior probability before starting the investigation, but it has several contradictions. What is the probability that an innocent person's alibi gets contradicted? What is the probability that a guilty person has a motive? All these things matter in a investigation. So we can't obviously use this method to classify the tasks. Meanwhile, additive scoring is completely [transparent](#). If we mention that a contradiction is worth 20 points because it's the strongest signal .. it

means someone is lying about where they were. A motive is worth 15 points because it's necessary but not sufficient .. lots of people have motive and didn't do it. No alibi is worth 10 points because it's weak evidence.. absence of proof isn't proof. As per instruction, this method is auditable and we can give a proper reasoning for allotting particular number of points to a particular contradiction/motive.

The Three-Phase Loop :

The real life investigations always work in a phase [loop](#). We generally don't investigate a person deeply unless and until we have a baseline information from all the suspects.

So similarly, I chose to set 3 phases.

Phase1: In this you ask every suspect the same three questions. Where were you, what was your relationship with the victim, did you notice anything. This gives you a baseline of facts to compare.

Phase2: It is basically contradiction hunting. Now that I have facts from everyone, test it over the set of the rules which I have provided [.If](#) we find a contradiction, we approach that person directly and now this creates a new testimony, which generates new facts and now might create set of new conflicts or contradictions

Phase3: After scoring the different suspects, we take top 2 suspects by score and then ask them deeper questions. Did you have reason to want the victim dead, can anyone confirm your alibi, who do you think did it. The last question is particularly useful - innocent suspects often point to who they're suspicious of, which can corroborate your reasoning.

Judging the Final Victim:

For the final victim to be found out, I set a criteria. If the top scorer's score is at least 1.5 times the score of the 2nd position suspect then the first person is declared as a victim and they have a confirmed motive and broken alibi.

Then I added a priority queue to make targeted questioning evidence driven.

Formula: $\text{priority} = (\text{contradictions} * 25) + (\text{motive} * 20 : 0) + (\text{no alibi} * 15 : 0) - (\text{questions asked} * 3)$

The 3 questions have been subtracted because it prevents the system from endlessly hammering the same suspect while others go unquestioned. It forces exploration. I have kept contradiction weight the highest because it is the strongest signal.